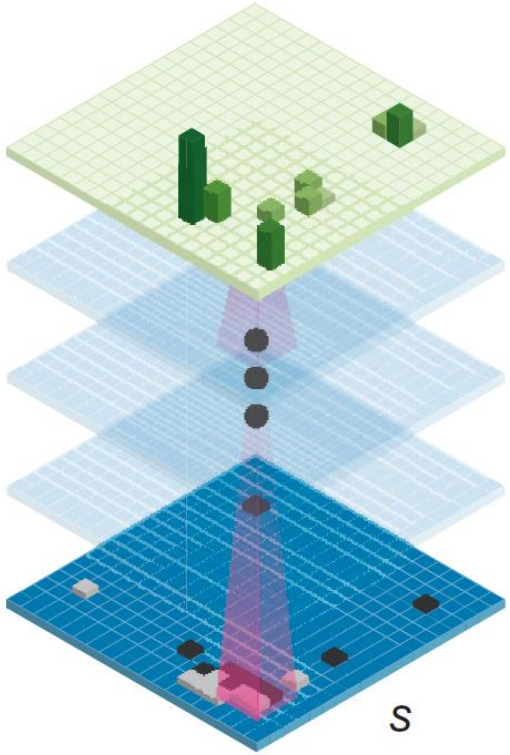


第5章 树形结构

目录



第一节 树

第二节 二叉树

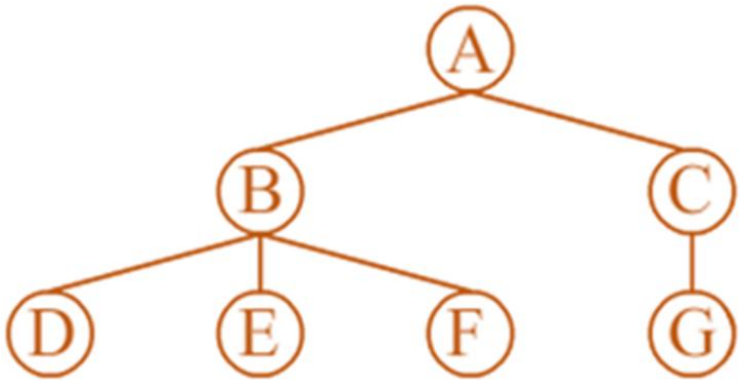
第三节 哈夫曼树及哈夫曼编码

第四节 树和森林

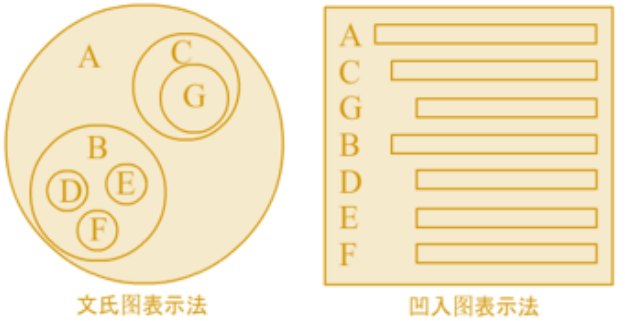
第一节 什么是树

1.1 树的基本概念

人们在生活中所见的家谱、文件系统等，虽然表现形式各异，在本质上是树结构。



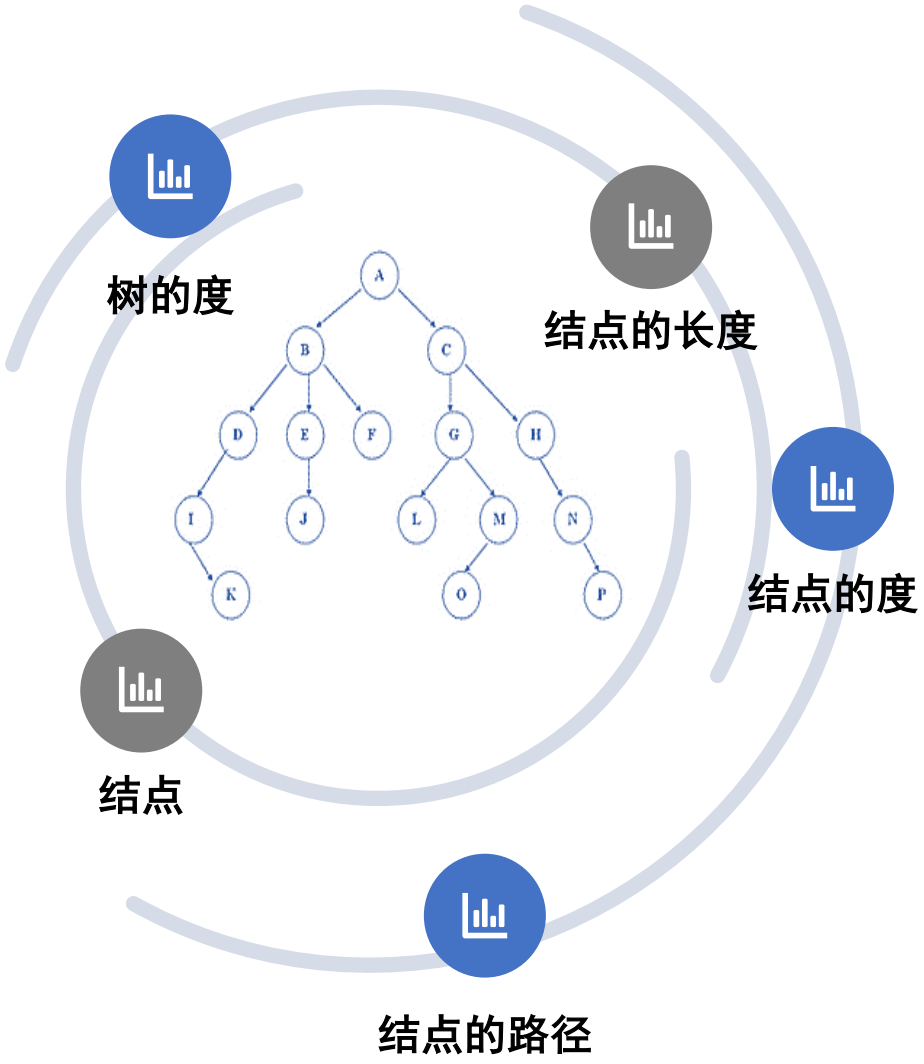
树的表示方法有多种，如树形表示法、文氏图表示法、凹入图表示法和广义表表示法。



A(B(D,E,F),C(G))

广义表表示法

1.2 树的术语



结点

构成树的数据元素，就是其他数据结构中存储的数据项，在树形表示法中用圆圈表示。

结点的路径

从根结点到该结点所经过结点的顺序排列。

路径的长度

路径中包含的分支数。

结点的度

结点拥有的子树的数目。

树的度

树中所有结点的度的最大值。

1.2 树的术语

叶结点

树中度为0的结点，也叫终端结点。

分支结点

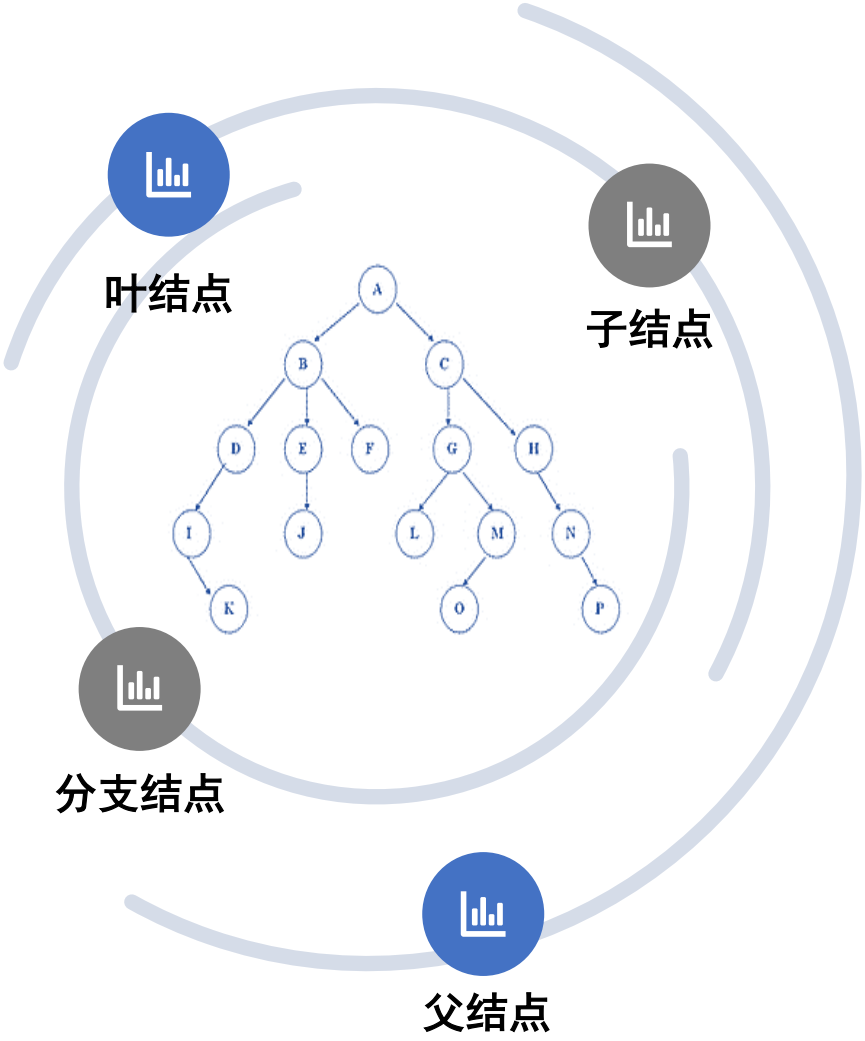
树中度不为0的结点，也叫非终端结点。

子结点

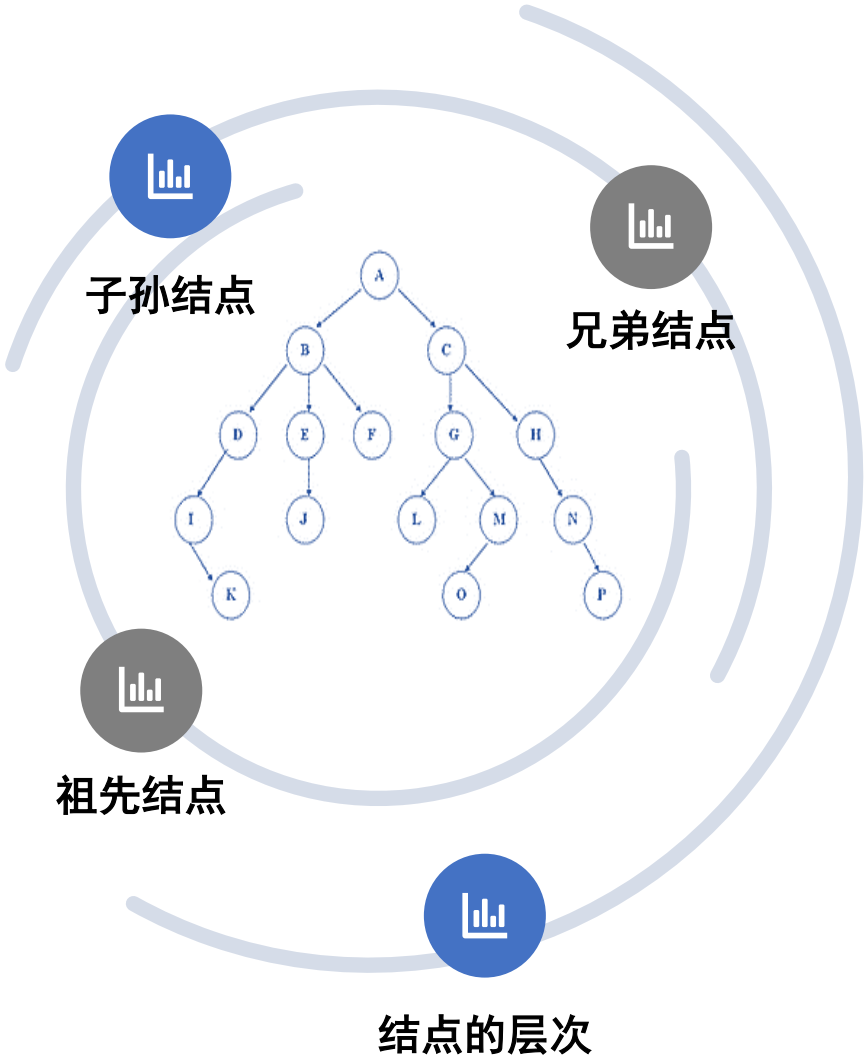
结点的子树的根结点，也叫孩子结点。

父结点

具有子结点的结点叫该子结点的父结点，也叫双亲结点。



1.2 树的术语



子孙结点

结点的子树中的任意结点。

祖先结点

结点的路径中除自身之外的所有结点。

兄弟结点

和结点具有同一父结点的结点。

结点的层次

树中根结点的层次为0，其他结点的层次是父结点的层次加1。

1.2 树的术语

树的深度

树中所有结点的层次数的最大值加1。

有序树

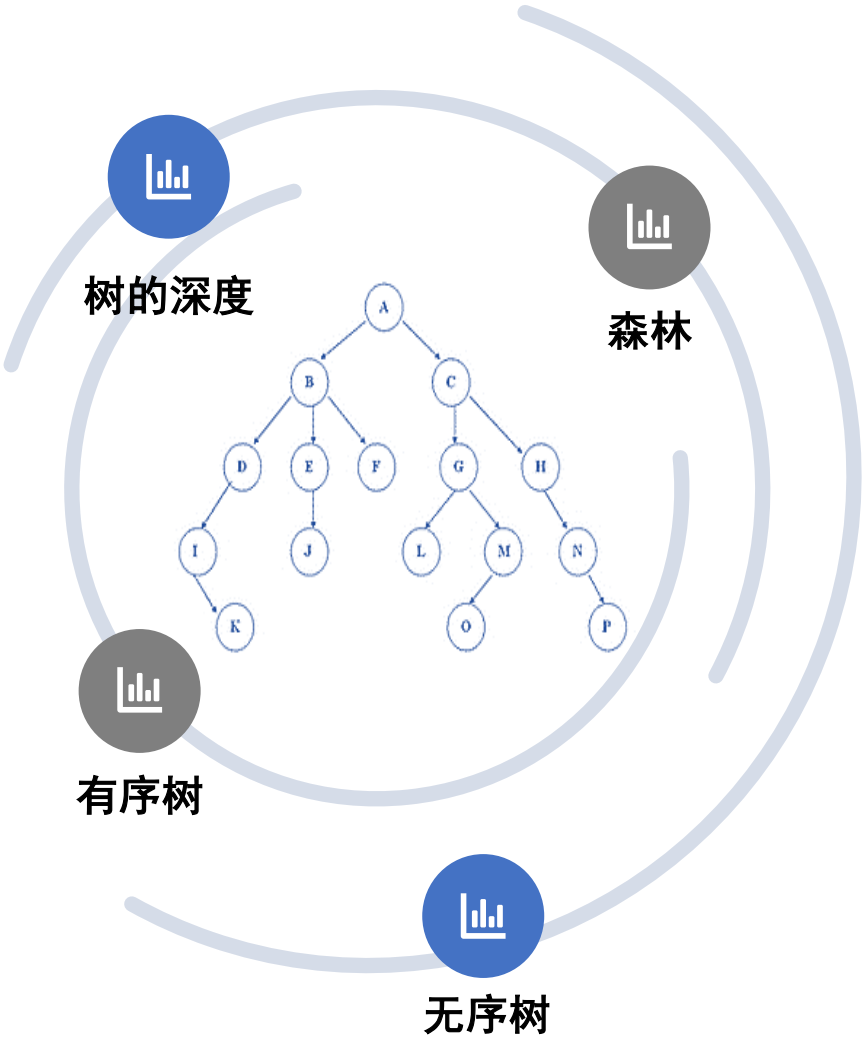
树的各结点的所有子树具有次序关系，不可以改变位置。

无序树

树的各结点的所有子树之间无次序关系，可以改变位置。

森林

由多个互不相交的树构成的集合。给森林加上一个根结点就变成一棵树，将树的根结点删除就变成森林。



第二节 二叉树

2.1 二叉树的基本概念

- 1. 普通二叉树

二叉树是特殊的有序树，它也是由n个结点构成的有限集合。当n=0时称为空二叉树。二叉树的每个结点最多只有两棵子树，子树也为二叉树，互不相交且有左右之分，分别称为左二叉树和右二叉树。

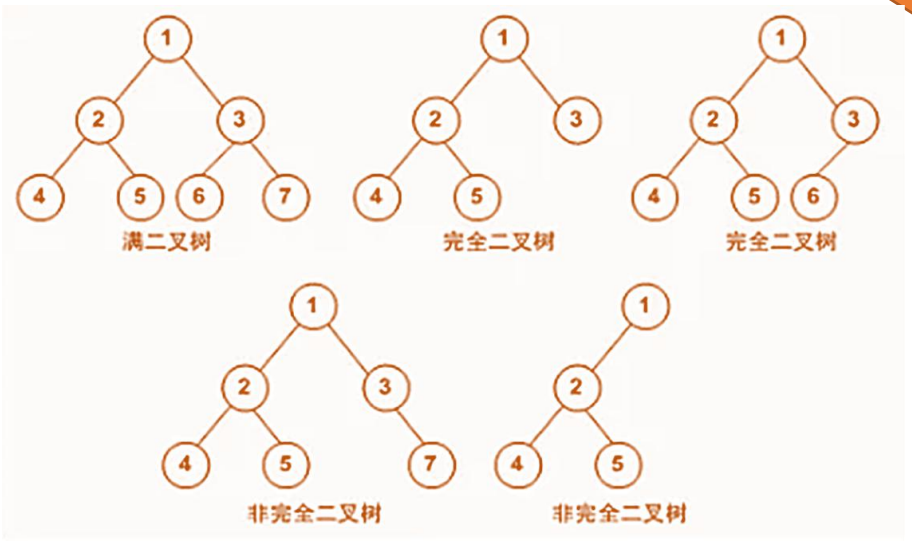
二叉树也是递归定义的，在树中定义的度、层次等术语同样也适用于二叉树。

- 2. 满二叉树

满二叉树是特殊的二叉树，它要求除叶结点外的其他结点都具有两棵子树，并且所有的叶结点都在同一层上。

- 3. 完全二叉树

完全二叉树是特殊的二叉树，若完全二叉树具有n个结点，它要求n个结点与满二叉树的前n个结点具有完全相同的逻辑结构。



2.2 二叉树的性质

- 性质1： 二叉树中第i层的结点数最多为2i。

证明： 当i=0时只有一个根结点，成立； 假设对所有的k(0≤k<i)成立，即第i-1层上最多有2^{i-1}个结点，那么由于每个结点最多有两棵子树，在第i层上结点数最多为$2^{i-1}\times 2=2^{i}$个，得证。

- 性质2： 深度为h的二叉树最多有2h-1个结点。

证明： 由性质1得，深度为h的二叉树的结点个数最多为20+21+...+2h-1=2h-1，得证。

- 性质3： 若二叉树的叶结点的个数为n，度为2的结点个数为m，有n=m+1。

证明： 设二叉树中度为1的结点个数为k，二叉树的结点总数为s，有s=k+n+m。又因为除根结点外每个结点都有一个进入它的分支，所以s-1=k+2*m。整理后得到n=m+1，得证。

2.2 二叉树的性质

- 性质4： 具有n个结点的完全二叉树，其深度为 $\lfloor log_2 n \rfloor + 1$ 或者$\lceil log_2(n+1) \rceil$。

证明： 设此二叉树的深度为h，由性质2可得2h-1≤n<2h，两边取对数，可得 h-1≤log2n<h，因为h为整数，所以h=$\lfloor \log_2 n \rfloor + 1$，得证。

- 性质5： 具有n个结点的完全二叉树，从根结点开始自上而下、从左向右对结点从0开始编号。对于任意一个编号为i的结点：

证明：(1) 若i=0，结点为根结点，没有父结点； 若i>0，则父结点的编号为$\lfloor (i-1)/2 \rfloor$。

(2) 若2i+1≥n，该结点无左孩子，否则左孩子结点的编号为2i+1。

(3) 若2i+2≥n，该结点无右孩子，否则右孩子结点的编号为2i+2。

2.2 二叉树的性质——例题

【例5.1】对于任意一个满二叉树，其分支数B＝2(n_0-1)，其中n_0为终端结点数。

解： 设n2为度为2的结点，因为在满二叉树中没有度为1的结点，所以有

$$\mathbf{n} = \boldsymbol{n_0} + \boldsymbol{n_2}$$

设B为树中分支数，则

$$\mathbf{n} = \boldsymbol{B} + \mathbf{1}$$

所以

$$\mathbf{B} = \boldsymbol{n_0} + \boldsymbol{n_2} - \mathbf{1}$$

再由二叉树的性质

$$\boldsymbol{n_0} = \boldsymbol{n_2} + \mathbf{1}$$

代入上式有

$$\mathbf{B} = \boldsymbol{n_0} + \boldsymbol{n_0} - \mathbf{1} - \mathbf{1} = \mathbf{2}(\boldsymbol{n_0} - \mathbf{1})$$



2.2 二叉树的性质——例题

【例5.2】已知一棵度为m的树中有n_1个度为1的结点、n_2个度为2的结点、…、n_m个度为m的结点，问该树中共有多少个叶子结点？

解： 设该树的总结点数为n，则

$$n = n_0 + n_1 + n_2 + \cdots + n_m$$

又

$$n = \text{分支数} + 1$$

$$= 0 \times n_0 + 1 \times n_1 + 2 \times n_2 + \cdots + m \times n_m + 1$$

由上述两式可得

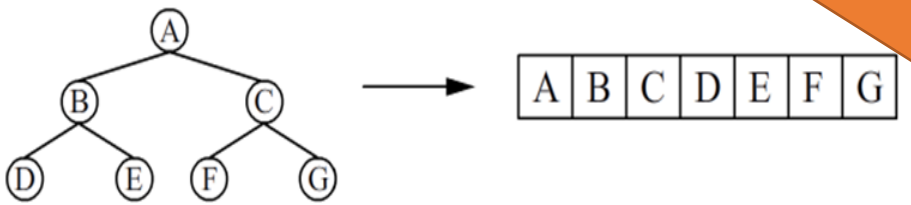
$$n_0 = n_2 + 2n_3 + \cdots + (m-1)n_m + 1$$



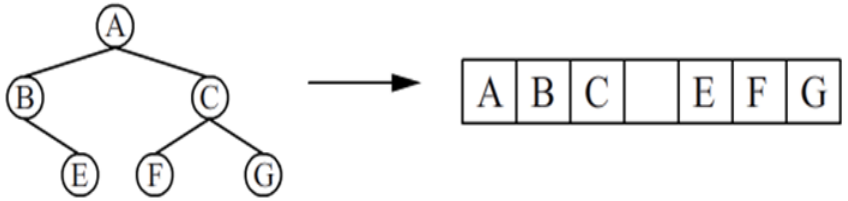
2.3 二叉树的存储结构

（1）二叉树的顺序存储结构

- 二叉树的顺序存储结构是指将二叉树的各个结点存放在一组地址连续的存储单元中，所有结点按结点序号进行顺序存储。因为二叉树为非线性结构，所以必须先将二叉树的结点排成线性序列再进行存储，实际上是对二叉树先进行一次层次遍历。二叉树的各结点间的逻辑关系由结点在线性序列中的相对位置确定。
- 可以利用2.2节中的性质5将二叉树的结点排成线性序列，将结点存放在下标为对应编号的数组元素中。为了存储非完全二叉树，需要在树中添加虚结点使其成为完全二叉树后再进行存储，这样会造成存储空间的浪费。



完全二叉树的顺序储存



非完全二叉树的顺序储存

2.3 二叉树的存储结构

（2）二叉树的链式存储结构

二叉树的链式存储结构是指将二叉树的各个结点随机存放在存储空间中，二叉树的各结点间的逻辑关系由指针确定。每个结点至少要有两条链分别连接左、右孩子结点才能表达二叉树的层次关系。根据指针域个数的不同，二叉树的链式存储结构又分为以下两种：

- a.二叉链式存储结构

二叉树的每个结点设置两个指针域和一个数据域。数据域中存放结点的值，指针域中存放左、右孩子结点的存储地址。

采用二叉链表存储二叉树，每个结点只存储了到其孩子结点的单向关系，没有存储到其父结点的关系，因此要获得父结点将花费较多的时间，需要从根结点开始在二叉树中进行查找，所花费的时间是遍历部分二叉树的时间，且与查找结点所处的位置有关。

- b. 三叉链式存储结构

二叉树的每个结点设置3个指针域和一个数据域。数据域中存放结点的值，指针域中存放左、右孩子结点和父结点的存储地址。

2.3 二叉树的存储结构

（2）二叉树的链式存储结构

下图所示为二叉链式存储和三叉链式存储的结点结构。

lchild	data	rchild

二叉链表结点

parent	lchild	data	rchild

三叉链表结点

两种链式存储结构各有优缺点，二叉链式存储结构空间利用率高，而三叉链式存储结构既便于查找孩子结点，又便于查找父结点。在实际应用中，二叉链式存储结构更加常用，因此本书中二叉树的相关算法都是基于二叉链式存储结构设计的。



2.3 二叉树的存储结构

（3）二叉链式存储结构的节点类的描述

```
class BiTreeNode(metaclass=ABCMeta):
    def __init__(self,data=None,lchild=None,rchild=None):
        self.data = data # 数据域的值
        self.lchild = lchild # 左孩子的指针
        self.rchild = rchild # 右孩子的指针
```

2.3 二叉树的存储结构

（4）二叉树类的描述

```
class BiTree(object):
        def __init__(self,root=None):
                self.root = root # 二叉树的根节点
```

2.4 二叉树的遍历



定义：

二叉树的遍历是指沿着某条搜索路径访问二叉树的结点，每个结点被访问的次数有且仅有一次。

2.4 二叉树的遍历

（1）二叉树的遍历方法

二叉树通常可划分为3个部分，即根结点、左子树和右子树。根据3个部分的访问顺序不同，可将二叉树的遍历方法分为以下几种。

a. 层次遍历

自上而下、从左到右依次访问每层的结点。

b. 先序遍历

先访问根结点，再先序遍历左子树，最后先序遍历右子树。

c. 中序遍历

先中序遍历左子树，再访问根结点，最后中序遍历右子树。

d. 后序遍历

先后序遍历左子树，再后序遍历右子树，最后访问根结点。



2.4 二叉树的遍历

（2）二叉树遍历操作实现的递归算法

前序遍历

```
def preOrder(root):
    if root is not None:
        print(root.data,end=' ')
        BiTree.preOrder(root.lchild)
        BiTree.preOrder(root.rchild)
```

中序遍历

```
def inOrder(root):
    if root is not None:
        BiTree.inOrder(root.lchild)
        print(root.data,end=' ')
        BiTree.inOrder(root.rchild)
```

后序遍历

```
def postOrder(root):
    if root is not None:
        BiTree.postOrder(root.lchild)
        BiTree.postOrder(root.rchild)
        print(root.data,end=' ')
```

2.4 二叉树的遍历

（3）二叉树遍历操作实现的非递归算法

二叉树遍历操作的递归算法结构简洁，易于实现，但是在时间上开销较大，运行效率较低，为了解决这一问题，可以将递归算法转换为非递归算法，转换方式有以下两种：

a.使用临时遍历保存中间结果，用循环结构代替递归过程；

b.利用栈保存中间结果。

二叉树遍历操作实现的非递归算法利用栈结构通过回溯访问二叉树的每个结点。



2.4 二叉树的遍历

（3）二叉树遍历操作实现的非递归算法

A. 先序遍历

先序遍历从二叉树的根结点出发，沿着该结点的左子树向下搜索，每遇到一个结点先访问该结点，并将该结点的右子树入栈。先序遍历左子树完成后再从栈顶弹出右子树的根结点，然后采用相同的方法先序遍历右子树，直到二叉树的所有结点都被访问。其主要步骤如下:

- (1) 将二叉树的根结点入栈。
- (2) 若栈非空，将结点从栈中弹出并访问。
- (3) 依次访问当前访问结点的左孩子结点，并将当前结点的右孩子结点入栈。
- (4) 重复步骤(2)和(3)，直到栈为空。

```
1  def preOrder2(root):
2          p = root
3          s = LinkStack()
4          s.push(p)
5          while not s.isEmpty():
6                  p = s.pop()
7                  print(p.data,end=' ')
8                  while p is not None:
9                          if p.lchild is not None:
10                                 print(p.lchild.data,end=' ')
11                         if p.rchild is not None:
12                                 s.push(p.rchild)
13                         p = p.lchild
```

2.4 二叉树的遍历

（3）二叉树遍历操作实现的非递归算法

B. 中序遍历

中序遍历从二叉树的根结点出发，沿着该结点的左子树向下搜索，每遇到一个结点就使其入栈，直到结点的左孩子结点为空。再从栈顶弹出结点并访问，然后采用相同的方法中序遍历结点的右子树，直到二叉树的所有结点都被访问。其主要步骤如下:

- (1) 将二叉树的根结点入栈。
- (2) 若栈非空，将栈顶结点的左孩子结点依次入栈，直到栈顶结点的左孩子结点为空。
- (3) 将栈顶结点弹出并访问，并使栈顶结点的右孩子结点入栈。
- (4) 重复步骤(2)和(3)，直到栈为空。

```
1  def inOrder2(root):
2          p = root
3          s = LinkStack()
4          s.push(p)
5          while not s.isEmpty():
6                  while p.lchild is not None:
7                          p = p.lchild
8                          s.push(p)
9                  p = s.pop()
10                 print(p.data,end=' ')
11                 if p.rchild is not None:
12                         s.push(p.rchild)
```

2.4 二叉树的遍历

（3）二叉树遍历操作实现的非递归算法

C. 后序遍历

后序遍历从二叉树的根结点出发，沿着该结点的左子树向下搜索，每遇到一个结点需要判断其是否为第一次经过，若是则使结点入栈，后序遍历该结点的左子树，完成后再遍历该结点的右子树，最后从栈顶弹出该结点并访问。后序遍历算法的实现需要引入两个变量，一个为访问标记变量flag，用于标记栈顶结点是否被访问，若flag=true，证明该结点已被访问，其左子树和右子树已经遍历完毕，可继续弹出栈顶结点，否则需要先遍历栈顶结点的右子树； 一个为结点指针t,指向最后一个被访问的结点，查看栈顶结点的右孩子结点，证明此结点的右子树已经遍历完毕，栈顶结点可出栈并访问。其主要步骤如下:

- (1) 将二叉树的根结点入栈，t赋值为空。
- (2) 若栈非空，将栈顶结点的左孩子结点依次入栈，直到栈顶结点的左孩子结点为空。
- (3) 若栈非空，查看栈顶结点的右孩子结点，若右孩子结点为空或者与p相等，则弹出栈顶结点并访问，同时使t指向该结点，并置flag为true； 否则将栈顶结点的右孩子结点入栈，并置flag为false。
- (4) 若flag为true，重复步骤(3)； 否则重复步骤(2)和(3)，直到栈为空。

```
1  def postOrder2(root):
2          p = root
3          t = None
4          flag = True
5          s = LinkStack()
6          if p is not None:
7                  s.push(p)
8                  while p.child is None:
9                          p = p.lchild
10                         s.push(p)
11                 while not s.isEmpty() and flag:
12                         if p.rchild == t or p.rchild is None:
13                                 print(p.data,end=' ')
14                                 flag = True
15                                 t = p
16                                 s.pop()
17                         else:
18                                 s.push(p.rchild)
19                                 flag = False
```

2.5 二叉树遍历算法的应用

（1）二叉树上的查找算法

二叉树上的查找是在二叉树中查找值为x的结点，若找到返回该结点，否则返回空值，可以在二叉树的先序遍历过程中进行查找，主要步骤如下：

(1) 若二叉树为空，则不存在值为x的结点，返回空值； 否则将根结点的值与x进行比较，若相等，返回该结点。

(2) 若根结点的值与x的值不等，则在左子树中进行查找，若找到，则返回该结点。

(3) 若没有找到，则在根结点的右子树中进行查找，若找到，返回该结点，否则返回空值。



2.5 二叉树遍历算法的应用

（1）二叉树上的查找算法

```
def searchNode(t,x):
        if t is None:
                return None
        if t.data == x:
                return t
        else:
                lresult = searchNode(t.lchild,x)
        if lresult == None:
                return searchNode(t.rchild,x)
        else:
                return lresult
```



2.5 二叉树遍历算法的应用

（2）统计二叉树的结点个数的算法

二叉树的结点个数等于根结点加上左、右子树的结点的个数，可以利用二叉树的先序遍历序列，引入一个计数变量count，count的初值为0，每访问根结点一次就将count的值加1，其主要操作步骤如下：

(1) count值初始化为0。

(2) 若二叉树为空，返回count值。

(3) 若二叉树非空，则count值加1，
统计根结点的左子树的结点个数，
并将其加到count中；
统计根结点的右子树的结点个数，
并将其加到count中。

```
def nodeCount(t):
        count = 0
        if t is not None:
                count += 1
                count += nodeCount(t.lchild)
                count += nodeCount(t.rchild)
        return count
```

2.5 二叉树遍历算法的应用

（3）求二叉树的深度

二叉树的深度是所有结点的层次数的最大值加1，也就是左子树和右子树的深度的最大值加1，可以采用后序遍历的递归算法解决此问题，其主要步骤如下：

(1) 若二叉树为空，返回0。

(2) 若二叉树非空，

求左子树的深度、求右子树的深度。

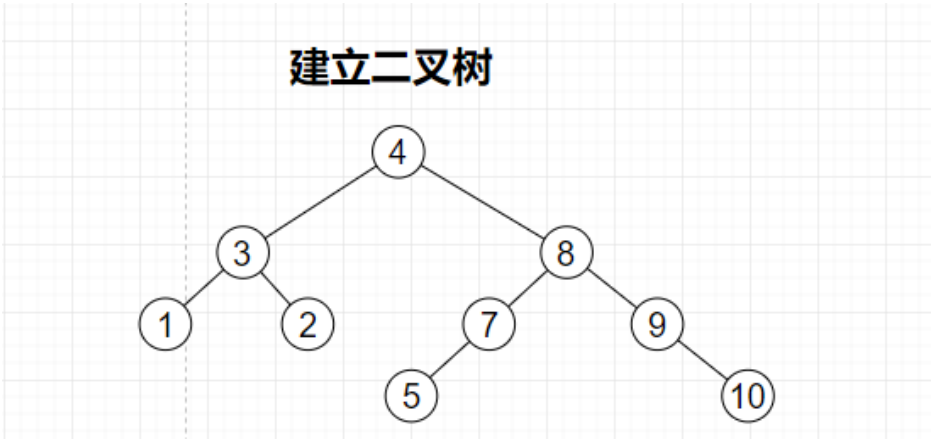
(3) 比较左、右子树的深度，

取最大值加1即为二叉树的深度。

```
def getDepth(t):
        if t is None:
                return 0
        ldepth = getDepth(t.lchild)
        rdepth = getDepth(t.rchild)
        if ldepth < rdepth:
                return rdepth + 1
        else:
                return ldepth + 1
```

2.6 二叉树的建立

二叉树遍历操作可使非线性结构的树转换成线性序列。先序遍历序列和后序遍历序列反映父结点和孩子结点间的层次关系，中序遍历序列反映兄弟结点间的左右次序关系。因为二叉树是具有层次关系的结点构成的非线性结构，并且每个结点的孩子结点具有左右次序，所以已知一种遍历序列无法唯一确定一棵二叉树，只有同时知道中序和先序遍历序列，或者同时知道中序和后序遍历序列，才能同时确定结点的层次关系和结点的左右次序，才能唯一确定一棵二叉树。





2.6 二叉树的建立

（1）由中序和先序遍历序列建立二叉树

其主要步骤为如下：

(1) 取先序遍历序列的第一个结点作为根结点，序列的结点个数为n。

(2) 在中序遍历序列中寻找根结点，其位置为i，可确定在中序遍历序列中根结点之前的i个结点构成的序列为根结点的左子树中序遍历序列，根结点之后的n-i-1个结点构成的序列为根结点的右子树中序遍历序列。

(3) 在先序遍历序列中根结点之后的i个结点构成的序列为根结点的左子树先序遍历序列，先序遍历序列之后的n-i-1个结点构成的序列为根结点的右子树先序遍历序列。

(4) 对左、右子树重复步骤(1)、(2)、(3)，确定左、右子树的根结点和子树的左右、子树。

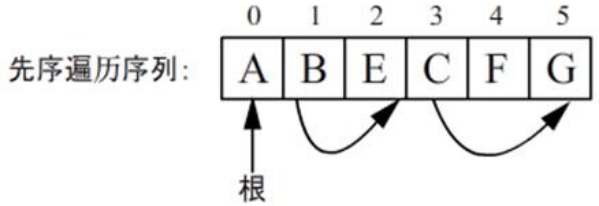
(5) 算法递归进行即可建立一棵二叉树。

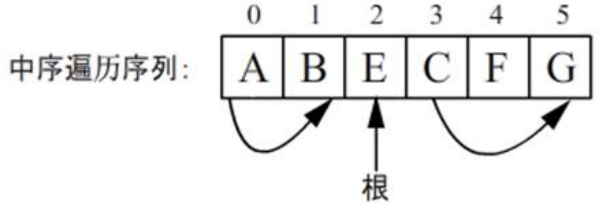


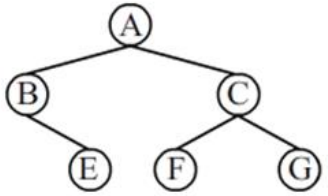
假设二叉树的先序遍历序列为ABECFG、

中序遍历序列为BEAFCG,

由中序和先序遍历序列建立二叉树的过程：







建立的二叉树

2.6 二叉树的建立

（1）由中序和先序遍历序列建立二叉树

算法如下：

```
def createBiTree(preOrder,inOrder,preo,ino,n):
    if n>0:
        i = 0
        c = preOrder.charAt(preo) # c 为先序序列的根结点
        while i<n:
            if inOrder.charAt(i+ino)==c:
                break
            i += 1
        root = BiTreeNode(c)
        root.lchild = createBiTree(preOrder,inOrder,preo+1,ino,i).root
        # 递归寻找左子树的根结点
        root.rchild = createBiTree(preOrder,inOrder,preo+i+1,ino+i+1,n-i-1).root
        # 递归寻找右子树的根结点
```

2.6 二叉树的建立

（2）由标明空子树的先序遍历序列创建二叉树

其主要步骤如下：

(1) 从先序遍历序列中依次读取字符。

(2) 若字符为#，建立空子树。

(3) 建立左子树。

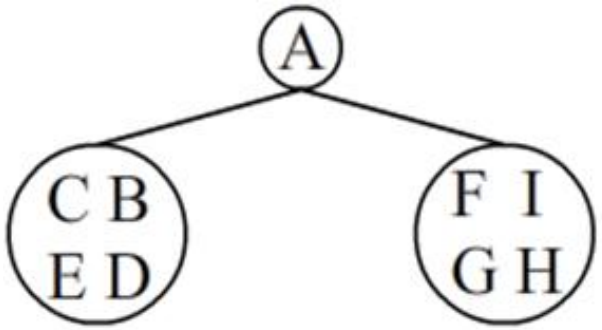
(4) 建立右子树。

```
def createBiTree(preOrder,i): # i 为常数 0
        c = preOrder.charAt(i) # 取字符
        if c != '#':
                root = BiTreeNode(c)
                i += 1
                root.lchild = createBiTree(preOrder,i).root
                i += 1
                root.rchild = createBiTree(preOrder,i).root
        else:
                root = None
```

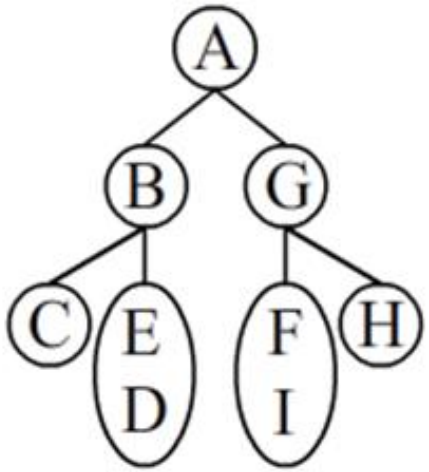
2.6 二叉树的建立——例题

【例5.3】已知二叉树的中序和后序序列分别为CBEDAFIGH和CEDBIFHGA，试构造该二叉树。

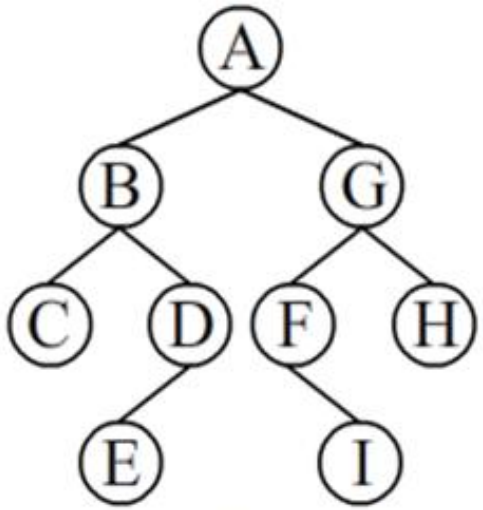
解： 二叉树的构造过程如下图所示。图(c)即为构造出的二叉树。



(a)第一步



(b)第二步



(c)第三步



第三节　哈夫曼树及哈夫曼编码

目前常用的图像、音频、视频等多媒体信息由于数据量大，必须对它们采用数据压缩技术来存储和传输。数据压缩技术通过对数据进行重新编码来压缩存储，以便减少数据占用的存储空间，在使用时再进行解压缩，恢复数据的原有特性。

其压缩方法主要有有损压缩和无损压缩两种。有损压缩是指压缩过程中可能会丢失数据信息，如将BMP位图压缩成JPEG格式的图像，会有精度损失； 无损压缩是指压缩存储数据的全部信息，确保解压后的数据不丢失。**哈夫曼编码**是数据压缩技术中的**无损压缩技术**。

3.1 哈夫曼树的基本概念

- **1. 结点间的路径**

结点间的路径是指从一个结点到另一个结点所经过的结点序列。从根结点到X结点有且仅 有一条路径。

- **2. 结点的路径长度**

结点的路径长度是指从根结点到结点的路径上的边数。

- **3. 结点的权**

结点的权是指人给结点赋予的一个具有某种实际意义的数值。

- **4. 结点的带权路径长度**

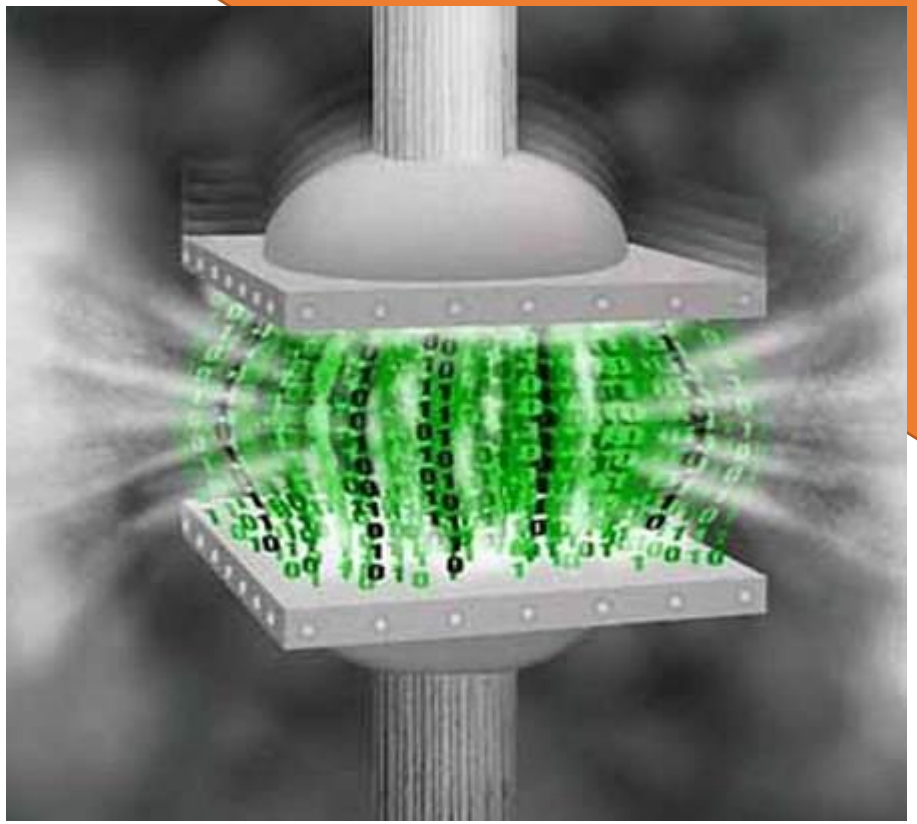
结点的带权路径长度是指结点的权值和结点的路径长度的乘积。

- **5. 树的带权路径长度**

树的带权路径长度是指树的叶结点的带权路径长度之和。

- **6. 最优二叉树**

最优二叉树是指给定n个带有权值的结点作为叶结点构造出的具有最小带权路径长度的二叉树。最优二叉树也叫哈夫曼树。



3.2 哈夫曼树的构造

给定n个叶结点，它们的权值分别是{w1,w2,…,wn}，构造相应的哈夫曼树的主要步骤如下：

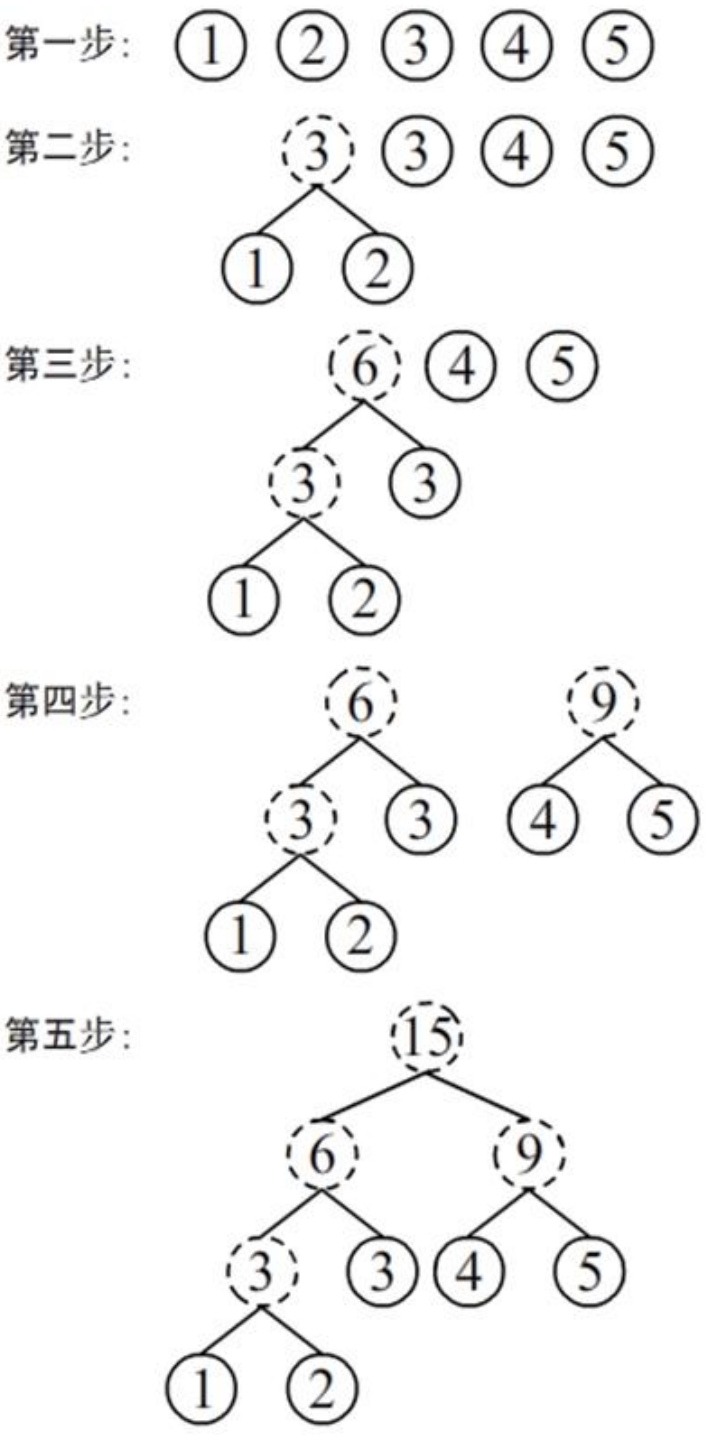
(1) 构造由n棵二叉树组成的森林，每棵二叉树只有一个根结点，根结点的权值分别为$\{w_1,w_2,\ldots,w_n\}$。

(2) 在森林中选取根结点权值最小和次小的两棵二叉树分别作为左子树和右子树去构造一棵新的二叉树，新二叉树的根结点权值为两棵子树的根结点权值之和。

(3) 将两棵二叉树从森林中删除，并将新的二叉树添加到森林中。

(4) 重复步骤(2)和(3)，直到森林中只有一棵二叉树，此二叉树即为哈夫曼树。

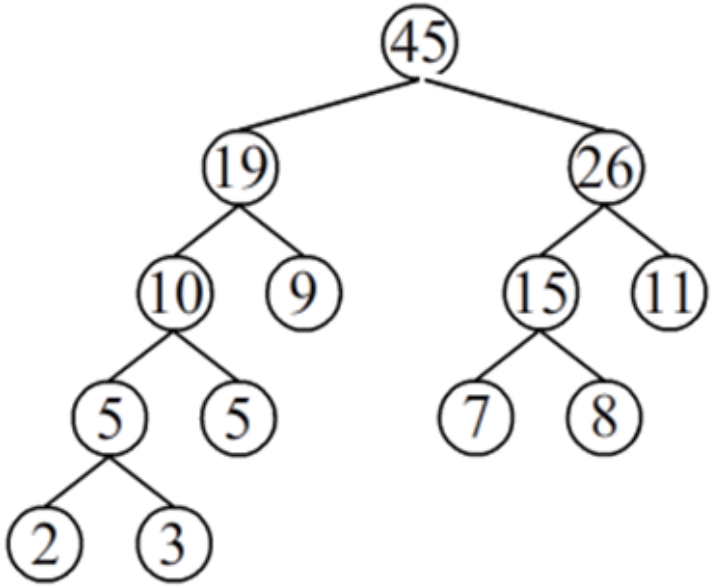
假设给定的权值为{1，2，3，4，5}，右图展示了哈夫曼树的构造过程。



3.2 哈夫曼树的构造

【例5.4】对于给定的一组权值W＝(5，2，9，11，8，3，7)，试构造相应的哈夫曼树，并计算它的带权路径长度。

解： 构造的哈夫曼树如下图所示：



树的带权路径长度如下：

WPL=2×4+3×4+5×3+7×3+8×3+9×2+11×2=120

3.3 哈夫曼编码

- 在传送信息时需要将信息符号转化成二进制组成的符号串，一般每个字符由一个字节或两个字节表示，即8或16个位数。为了提高存储和传输效率，需要设计对字符集进行二进制编码的规则，使得利用这种规则对信息进行编码时编码位数最小，即需要传输的信息量最小。

- 哈夫曼编码是一种变长的编码方案，数据的编码因其使用频率的不同而长短不一，使用频率高的数据其编码较短，使用频率低的数据其编码较长，从而使所有数据的编码总长度最短。各数据的使用频率通过在全部数据中统计重复数据的出现次数获得。

- 又因为在编码序列中若使用前缀相同的编码来表示不同的字符会造成二义性，额外的分隔符号会造成传输信息量的增加，为了省去不必要的分隔符号，要求每一个字符的编码都不是另一个字符的前缀，即每个字符的编码都是前缀编码。



3.3 哈夫曼编码

利用哈夫曼树构造出的哈夫曼编码是一种最优前缀编码

构造的主要步骤如下：

(1) 对于具有n个字符的字符集，将字符的频度作为叶结点的权值，产生n个带权叶结点。
(2) 根据上面章节中介绍的构造哈夫曼树的方法利用n个叶结点构造哈夫曼树。
(3) 根据哈夫曼编码规则将哈夫曼树中的每一条左分支标记为0、每一条右分支标记为1，则可得到每个叶结点的哈夫曼编码。

哈夫曼编码的译码过程是构造过程的逆过程，从哈夫曼树的根结点开始对编码的每一位进行判别，如果为0进入左子树，如果为1进入右子树，直到到达叶结点，即译出了一个字符。

3.4 构造哈夫曼树和哈夫曼编码的类的描述

构造哈夫曼树需要从子结点到父结点的操作，译码时需要从父结点到子结点的操作，所以为了提高算法的效率将哈夫曼树的结点设计为**三叉链式存储结构**。一个数据域存储结点的权值，一个标记域flag标记结点是否已经加入到哈夫曼树中，3个指针域分别存储着指向父结点、孩子结点的地址。

结点类的描述如下：

```
class HuffmanNode(object):
    def __init__(self,data,weight):
        self.data = data # 结点的值
        self.weight = weight # 结点的权值
        self.parent = None # 父结点
        self.lchild = None # 左孩子
        self.rchild = None # 右孩子
```

3.4 构造哈夫曼树和哈夫曼编码的类的描述

构造哈夫曼树算法，如下：

```
class HuffmanTree(object):
        def __init__(self,data):
                # data 是编码字符与出现次数的集合，例如 w = [('a',1),('b',2)]
                nodes = [ HuffmanNode(c,w) for c,w in data ]
                self.index = {} # 编码字符的索引
                while len(nodes)>1:
                        nodes = sorted(nodes,key=lambda x:x.weight)
                        s = HuffmanNode(None,nodes[0].weight+nodes[1].weight)
                        s.lchild = nodes[0]
                        s.rchild = nodes[1]
                        nodes[0].parent = nodes[1].parent = s
                        nodes = nodes[2:]
                        nodes.append(s)
                self.root = nodes[0]
                self.calIndex(self.root,'') # 递归计算每个字符的哈夫曼编码并保存

        def calIndex(self,root,str):
                if root.data is not None:
                        # 保存字符的编码
                        self.index[root.data] = str
                else:
                        self.calIndex(root.lchild,str+'0')
                        self.calIndex(root.rchild,str+'1')

        def queryHuffmanCode(self,c):
                if c not in self.index:
                        raise Exception("未编码的字符")
                return self.index[c]
```

3.4 构造哈夫曼树和哈夫曼编码的类的描述

【算法】若字符与出现频率对应关系如下：
[('a',5),('b',2),('c',9),('d',11),('e',8),('f',3),('g',7)]
求哈夫曼编码。

```
data = [('a',5),('b',2),('c',9),('d',11),('e',8),('f',3),('g',7)]


t = HuffmanTree(data)


for c,w in data:
        print('字符%s 的哈夫曼编码为：%s'%(c,t.queryHuffmanCode(c)))
```

3.4 构造哈夫曼树和哈夫曼编码的类的描述

【算法】若字符与出现频率对应关系如下：
[('a',5),('b',2),('c',9),('d',11),('e',8),('f',3),('g',7)]
求哈夫曼编码。

字符 a 的哈夫曼编码为：010
字符 b 的哈夫曼编码为：0110
字符 c 的哈夫曼编码为：00
字符 d 的哈夫曼编码为：10
字符 e 的哈夫曼编码为：111
字符 f 的哈夫曼编码为：0111
字符 g 的哈夫曼编码为：110

第四节　树和森林

4.1 树的存储结构

一棵树包含各结点间的**层次关系**和**兄弟关系**，两种关系的存储结构不同。

树的层次关系必须采用**链式存储结构存储**，通过链连接父结点和孩子结点。

一个结点的多个孩子结点(互称兄弟结点)之间是线性关系，可以采用**顺序存储结构或者链式存储结构**。

a. 树的父母孩子链表

树的父母孩子链表采用顺序存储结构存储多个孩子结点，其中children数组　存储多个孩子结点，各结点的children数组元素长度不同，为孩子个数。

b. 树的父母孩子兄弟链表

树的父母孩子兄弟链表采用链式存储结构存储多个孩子结点，结点中child链指向一个孩子结点，sibling链指向下一个兄弟结点。

4.1 树的存储结构

森林也可以使用父母孩子兄弟链表进行存储，这种存储结构实际上是把一棵树转换成一棵二叉树存储。其存储规则如下：

（1）每个结点采用child链指向其中一个孩子结点，多个孩子结点之间由sibling链连接起来，组成一条具有兄弟结点关系的单链表。

（2）将每棵树采用树的父母孩子兄弟链表存储。

（3）森林中的多棵树之间是兄弟关系，将这些树通过根的sibling链连接起来。



4.2 树的遍历规则

树的孩子优先遍历规则主要有两种，即先序遍历和后序遍历。
树的遍历规则也是递归的。

（1）树的先序遍历：访问根结点；按从左到右的次序遍历根的每一棵子树。
（2）树的后序遍历：按从左到右的次序遍历根的每一棵子树；访问根结点。

树的层次遍历规则同二叉树。

第五节 小结

小结



- (1) **树**是数据元素之间具有层次关系的非线性结构，是由n个结点构成的有限集合。与线性结构不同，树中的数据元素具有一对多的逻辑关系。
- (2) **二叉树**是特殊的有序树，它也是由n个结点构成的有限集合。当n=0时称为空二叉树。二叉树的每个结点最多只有两棵子树，子树也为二叉树，互不相交且有左、右之分，分别称为**左二叉树**和**右二叉树**。
- (3) **二叉树的存储结构**分为两种，即顺序存储结构和链式存储结构。二叉树的**顺序存储结构**是指将二叉树的各个结点存放在一组地址连续的存储单元中，所有结点按结点序号进行顺序存储； 二叉树的**链式存储结构**是指将二叉树的各个结点随机存放在存储空间中，二叉树的各结点间的逻辑关系由指针确定。

小结



- (4) 二叉树具有**先序遍历**、**中序遍历**、**后序遍历**和**层次遍历**4种遍历方式。
- (5) 最优二叉树是指给定n个带有权值的结点作为叶结点构造出的具有最小带权路径长度的二叉树，也叫**哈夫曼树**。

- (6) **哈夫曼编码**是数据压缩技术中的无损压缩技术，是一种变长的编码方案，使所有数据的编码总长度最短。

The End

谢谢观看